



Pratica 1 - Aula 3

🕒 Criado em 7 de setembro de 2026 11:29

☰ Tags

```
# achar o tempo médio das primitivas FOCA
from time import perf_counter
from matplotlib import pyplot as plt
import numpy as np

R = 100

lista_n = [1000, 10000, 100000]
t = []

for n in lista_n:
    tt = []

    for r in range(R):
        tic = perf_counter()

        for i in range(n):
            x = 1

        toc = perf_counter()
        tt.append(toc - tic)
```

```

# ordenar os tempos
tt = np.array(tt)
tt = np.sort(tt)

# testar valores de corte
for tcorte in tt[::-1]:
    tt_filtrado = tt[tt < tcorte]

    if len(tt_filtrado) > 1:
        media = np.mean(tt_filtrado)
        desvio = np.std(tt_filtrado)
        coef_variacao = desvio / media
        print(f"Coef de variação = {coef_variacao}")

        if coef_variacao < 0.15:
            break

print(f"n = {n}")
print(f"Tempo de corte = {tcorte}")
print(f"Média = {media}")
print(f"Desvio = {desvio}")
print(f"Coeficiente de variação = {coef_variacao}")
print(f"Quantidade de tempos = {len(tt_filtrado)}")
print()

t.append(tt_filtrado)

plt.hist(tt_filtrado, bins=50)
plt.title(f"n={n}")
plt.savefig(f"tempos.n{n}.png")
plt.show()

```

1. Qual é o objetivo desse código?

O comentário inicial já entrega:

```
# achar o tempo médio das primitivas FOCA
```

A ideia é:

Descobrir experimentalmente quanto tempo, em média, uma determinada quantidade de operações primitivas leva para ser executada.

Isso é importante porque, na aula anterior, vimos a fórmula:

$$T = N_O\tau_O + N_C\tau_C + N_A\tau_A$$

Só que existe uma pergunta:

De onde vêm?

É justamente isso que esse programa está tentando descobrir.

2. Primeiro: o professor está fazendo uma análise experimental

Na disciplina, existem três formas de análise:

Tipo	Ideia
Experimental	Executar o código e medir
Analítica	Calcular matematicamente
Semi-analítica	Combinar análise matemática e medições

Esse código está claramente trabalhando com a **experimental**.

Ele não está tentando deduzir o tempo apenas olhando para o código.

Ele está fazendo:

```
Código
↓
Executa
↓
Mede o tempo
↓
```

```
Repete várias vezes
↓
Filtra medições ruins
↓
Calcula média
↓
Obtém um tempo representativo
```

E isso é uma ideia muito importante para você guardar.

3. O que exatamente ele está medindo?

Veja:

```
for i in range(n):
    x = 1
```

O professor está criando propositalmente um código muito simples.

O objetivo não é fazer algo útil.

Ele quer **isolar uma primitiva**.

Nesse caso:

```
x = 1
```

é uma **atribuição (A)**.

Então podemos pensar:

```
for i in range(n):
    x = 1
    ↑
    primitiva que
    queremos medir
```

A ideia é executar essa atribuição muitas vezes e observar quanto tempo isso leva.

4. Por que usar vários valores de **n**?

Aqui:

```
lista_n = [1000, 10000, 100000]
```

Ele vai fazer o experimento para três tamanhos:

```
n = 1.000
n = 10.000
n = 100.000
```

Isso é importante porque não queremos medir apenas:

“Quanto tempo levou para executar?”

Queremos observar a relação entre:

quantidade de operações × tempo

Por exemplo, imagine que obtivéssemos:

n	tempo
1.000	0,001 s
10.000	0,010 s
100.000	0,100 s

Isso sugere uma relação aproximadamente linear.

Ou seja:

$$T(n) \propto n$$

Essa é justamente a ponte entre **medição experimental** e **análise de complexidade**.

5. Por que **R = 100** ?

Aqui:

```
R = 100
```

E depois:

```
for r in range(R):
```

Significa:

Para cada valor de `n`, o programa executará o experimento **100 vezes**.

Por quê?

Porque medir uma única vez é ruim.

O tempo de execução pode variar por diversos motivos:

- outros programas executando;
- sistema operacional;
- processos em segundo plano;
- gerenciamento de memória;
- variações do interpretador Python;
- etc.

Então:

```
1 medição
↓
pouca confiança

100 medições
↓
podemos observar a distribuição dos tempos
```

6. Como ele mede o tempo?

Aqui está uma parte nova e importante:

```
tic = perf_counter()
```

```
for i in range(n):
    x = 1

toc = perf_counter()
tt.append(toc - tic)
```

O `perf_counter()` fornece uma medida de tempo de alta resolução adequada para medir durações.

O raciocínio é simplesmente:

```
tic
↓
começa a medição

[executa o código]

↓
toc
fim da medição
```

Então:

$$tempo = toc - tic$$

Esse valor é colocado na lista:

```
tt.append(toc - tic)
```

Depois de 100 repetições, `tt` contém aproximadamente:

```
[tempo1, tempo2, tempo3, ..., tempo100]
```

7. Agora começa a parte estatística

Essa parte é provavelmente uma das coisas novas que você percebeu.

Depois das medições:

```
tt = np.array(tt)
tt = np.sort(tt)
```

Primeiro transforma a lista em um array do NumPy.

Depois ordena os tempos.

Imagine que tivéssemos:

```
0.001
0.001
0.002
0.002
0.002
0.003
0.020 ← estranho
```

Ordenar facilita analisar a distribuição.

E isso nos leva ao problema principal:

Nem toda medição é igualmente confiável.

Imagine que você está medindo um código que normalmente leva:

```
0,001 s
```

Mas em uma execução o sistema operacional ficou ocupado e apareceu:

```
0,020 s
```

O código **não ficou magicamente 20 vezes mais lento**.

Provavelmente aquela medição sofreu alguma interferência.

Então o professor precisa de uma maneira de **filtrar os valores anormais**.

8. O que é esse **tcorte** ?

Essa parte parece assustadora:


```
for tcorte in tt[::-1]:
```

Mas a ideia é simples.

`tt[::-1]` percorre o array **do maior para o menor valor**.

Então o professor está testando diferentes valores de corte.

Imagine:

tempos ordenados:

```
0.001
0.001
0.001
0.002
0.002
0.003
0.020
```

Ele começa considerando um corte próximo dos maiores valores e vai diminuindo.

O objetivo é descobrir:

Até onde posso considerar os tempos sem deixar os valores muito dispersos?

9. O filtro

Aqui:

```
tt_filtrado = tt[tt < tcorte]
```

Ele mantém somente os tempos menores que o valor de corte.

Por exemplo:

```
tcorte = 0.010
```

```
tempos:
0.001
0.001
0.002
```

```
0.002
0.003
0.020

↓ filtro

0.001
0.001
0.002
0.002
0.003
```

O `0.020` ficou de fora.

Isso é uma forma de **eliminar medições muito discrepantes**.

10. Depois ele calcula a média

Aqui:

```
media = np.mean(tt_filtrado)
```

É a média dos tempos que sobraram.

Matematicamente:

$$\bar{x} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

Então o professor está tentando encontrar um **tempo médio representativo da execução**.

11. Mas somente a média não basta

Essa é uma ideia muito importante.

Imagine dois experimentos:

Experimento A

```
10
10
10
10
10
```

Média:

10

Experimento B

```
1
5
10
15
19
```

Média:

10

Os dois possuem a mesma média.

Mas claramente o segundo experimento é muito mais instável.

Então precisamos saber:

▮ **Os valores estão próximos da média ou estão espalhados?**

É aí que entra o **desvio padrão**.

12. Desvio padrão

O código:

```
desvio = np.std(tt_filtrado)
```

O desvio padrão mede, de forma geral, **o quanto os valores se afastam da média**.

Se temos:

```
10
10
10
10
10
```

o desvio é muito pequeno.

Se temos:

```
2
5
10
18
25
```

o desvio é muito maior.

Então:

```
desvio pequeno
  ↓
medições mais consistentes

desvio grande
  ↓
medições mais dispersas
```

13. E aqui está uma das partes mais importantes

O professor calcula:

```
coef_variacao = desvio / media
```

Isso é o **coeficiente de variação**.

Matematicamente:

$$CV = \frac{\sigma}{\bar{x}}$$

Se fosse expresso em porcentagem:

$$CV(\%) = \frac{\sigma}{\bar{x}} \times 100$$

No código ele está usando a forma decimal.

Por exemplo:

```
CV = 0,10
```

equivale a:

```
10%
```

14. Por que usar o coeficiente de variação?

Porque ele permite avaliar a **dispersão em relação ao tamanho da própria média**.

O professor estabeleceu:

```
if coef_variacao < 0.15:  
    break
```

Ou seja:

$$CV < 0,15$$

ou:

$$CV < 15\%$$

Então o raciocínio é:

```
Mede 100 vezes
  ↓
Ordena
  ↓
Testa um corte
  ↓
Remove valores acima do corte
  ↓
Calcula média
  ↓
Calcula desvio padrão
  ↓
Calcula CV
  ↓
CV < 15%?
  ↙   ↘
 NÃO   SIM
  ↓     ↓
outro aceita
corte conjunto
```

15. Por que ele não simplesmente remove os maiores valores?

Porque isso seria meio arbitrário.

Em vez de dizer:

“Vou jogar fora os 10 maiores.”

ele está usando um **critério estatístico**:

“Vou procurar um conjunto de medições cuja variação seja menor que 15%.”

Isso torna o procedimento mais justificável.

16. O resultado final

Depois ele imprime:

```
print(f"n = {n}")
print(f"Tempo de corte = {tcorte}")
print(f"Média = {media}")
print(f"Desvio = {desvio}")
print(f"Coeficiente de variação = {coef_variacao}")
print(f"Quantidade de tempos = {len(tt_filtrado)}")
```

Então para cada n teremos algo conceitualmente parecido com:

```
n = 1000
Tempo de corte = ...
Média = ...
Desvio = ...
Coeficiente de variação = ...
Quantidade de tempos = ...
```

O que interessa para a análise é principalmente:

$n \rightarrow$ tempo médio

Ou seja:

```
n = 1.000      → tempo médio
n = 10.000     → tempo médio
n = 100.000    → tempo médio
```

Agora podemos estudar como o tempo cresce.

17. E por que fazer o histograma?

Essa parte:

```
plt.hist(tt_filtrado, bins=50)
```

gera um histograma dos tempos.

O objetivo é **visualizar a distribuição das medições**.

Então, em vez de apenas olhar:

```
média = 0.002  
desvio = 0.0001
```

you can visualize how the times are distributed.

É uma ferramenta auxiliar para entender o experimento.

18. A grande ideia da aula

Agora vem o ponto que eu acho que você deve realmente guardar.

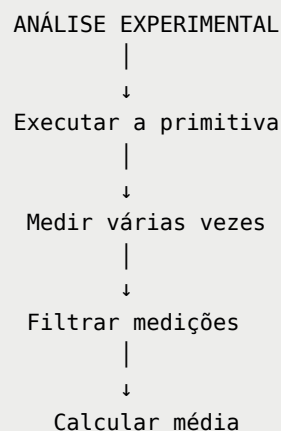
Na aula anterior tínhamos:

O problema era:

$$T(n) = N_O\tau_O + N_C\tau_C + N_A\tau_A$$

Como descobrir os valores de τ ?

Essa aula começa a responder:



|
↓
obter τ experimental

Então o programa não está simplesmente “calculando uma média”.

Ele está **calibrando experimentalmente o custo das primitivas**.

19. Isso conecta diretamente com a aula anterior

Lembra do:

$$T = N_O\tau_O + N_A\tau_A + N_C\tau_C$$

Agora imagine que o professor consiga experimentalmente descobrir:

$\tau_A = \text{algum valor}$

$\tau_O = \text{algum valor}$

$\tau_C = \text{algum valor}$

Então, quando encontrarmos um programa qualquer, podemos fazer:

1. analisar o código
↓
2. contar primitivas
↓
3. descobrir quantas vezes cada uma ocorre
↓
4. usar os τ obtidos experimentalmente
↓
5. estimar o tempo

Isso é **muito mais próximo do objetivo real da disciplina**.

20. E tem uma sutileza importante nesse código

Existe uma coisa que eu não quero que você confunda.

O professor escreve:

```
for i in range(n):  
    x = 1
```

e parece que estamos medindo **somente** `x = 1`.

Mas tecnicamente o cronômetro engloba também o gerenciamento do `for`:

```
tic  
↓  
for i in range(n):  
    x = 1  
↓  
toc
```

Portanto, o tempo medido é do **trecho inteiro**, não exclusivamente da atribuição.

A interpretação do experimento depende da forma como o professor está tratando e descontando/considerando o custo do laço.

Isso é uma coisa que vale observar nas próximas aulas, porque provavelmente é justamente daí que virão outros métodos ou códigos de calibração.

21. O que é material 2 e o que é novidade aqui?

Eu separaria assim:

Já vimos

- primitivas;
- FOCA;
- laços;

- quantidade de operações;
- função de tempo;
- τ_O, τ_C, τ_A τ_O, τ_C, τ_A ;
- relação entre operações e tempo;
- análise experimental como uma das formas de análise.

Novo nesta aula

- `perf_counter()` ;
- repetição de experimentos;
- distribuição dos tempos;
- ordenação das medições;
- filtragem;
- média;
- desvio padrão;
- coeficiente de variação;
- critério de corte de 15%;
- histograma;
- **calibração experimental das primitivas.**

E eu diria que **essa última é a mais importante conceitualmente.**

O raciocínio inteiro da aula

Se eu tivesse que transformar esse código em uma única linha de raciocínio para você estudar:

O professor quer descobrir experimentalmente quanto tempo uma primitiva leva. Para isso, executa uma quantidade conhecida de operações várias vezes, mede os tempos, elimina medições muito discrepantes, verifica a estabilidade dos resultados através do coeficiente de variação e usa a média das medições restantes como uma estimativa do tempo da operação.

E isso depois alimenta a análise que vocês já começaram:

Código $\rightarrow$ Primitivas $\rightarrow$ Contagem $\rightarrow \tau \rightarrow T(n) \rightarrow$ Complexidade